Module_Recon Clarification Register

Last Updated: 2026-11-12

Clarification questions for `Review.md` before the Module_Recon refactor is written as a SpecKit spec. No code changes are implied by this document. Answer the questions in Section 11 (or reply inline) and the answers become the "Confirmed Clarifications" section of the new spec.

How To Use This Document

- Every question has a stable ID (`SC` , `AR` , `UI` , `LG` , `DS` , `TS` , `MG` , `NF`).
- `Recommendation` is the default the spec will assume when no answer is supplied.
- Section 10 lists every recommendation in one place, so a single "accept all recommendations" reply unblocks spec authoring.
- Questions marked **BLOCKER** change the data model, the write path, or the algorithm shape. Those cannot be defaulted safely.

Contents

1. Ground truth already in the repo
2. Scope and module boundary (SC)
3. Scaffolding and architecture (AR)
4. UI and UX (UI)
5. Reconciliation logic (LG)
6. Data sources and schema (DS)
7. Testing (TS)
8. Migration and cleanup (MG)
9. Non-functional (NF)
10. Recommended defaults
11. Decision record

1. Ground Truth Already In The Repo

Read this before answering anything. Several `(VERIFY)` items in `Review.md` are already answered by the existing implementation, and several assumptions in `Review.md` conflict with what is actually built.

Area	What exists today	Consequence for the refactor
Service	`Module_Receiving/Services/Service_ReceivingLocationReconciliation.cs` (~1.2k lines) implements transfer-movement-based allocation	Review.md's algorithm is a *simplification* of what is already implemented
Contract	`Module_Receiving/Contracts/IService_ReceivingLocationReconciliation.cs` — 4 methods: `GetRecommendedLocationsAsync` , `PreviewLocationsAsync` , `ApplyLocationUpdateAsync` , `ReconcileLocationsAsync`	`GetRecommendedLocationsAsync` is also called by `ViewModel_Receiving_LoadEntry` , so the interface cannot move wholesale
Models	`Model_ReceivingLocationReconciliationItem` , `Model_ReceivingLocationReconciliationSummary` (both in `Module_Receiving/Models`)	Move candidates, but they reference `Model_ReceivingLoad`
Grouping key	`(PartID, ReceivedDate.Date)` — PO is **not** part of the group key	Review.md asks for `(PO, Part)` grouping. This is a real behavioral conflict (see `LG-01`)

Area	What exists today	Consequence for the refactor
Date scoping	No user date range. `includeAllHistory` bool: `StartDate = 2000-01-01` or `DateTime.Today`, `EndDate = Today + 1`	A user-selected range is genuinely new
Transfer discovery	`Service_InforVisualConnect` → query `18` with a **±2 day** window around each group's receipt date	Not a range; an anchor-date window
Strict validation	Allocated rows per destination bucket must exactly equal transfer quantity for `receiptDate` and `receiptDate + 1`	Different gate than Review.md's receiver-quantity equality (`LG-04`)
Cross-pass merge	Current Labels + History rows for the same `(part, date)` are already reconciled together (documented Fix 3 in `docs/prompts/reconciliation-location-fix.md`)	Preserve; do not "fix" back to separate passes
Allocation	Deterministic exact-fit then best-fit, ordered by quantity desc → load number → load id	Ordering is a documented invariant (do not change silently)
Visual "receiver"	**No** `RECEIVER` / `RECEIVER_LINE` query exists anywhere in the app	Review.md §5.1 has no implementation behind it (`LG-02`)
Current-inventory compare	Not used in the preview path. Only `GetRecommendedLocationsAsync` reads evidence query `16`	Review.md §5.2 "compare with current Visual locations" is not implemented in the reconcile flow (`LG-03`)
MySQL write path	`IService_MySQL_Receiving.UpdateCurrentLabelDataAsync` → `sp_Receiving_LabelData_Update`; `UpdateReceivingLoadsAsync` → `Dao_ReceivingLoad.UpdateLoadsAsync`	SP-only writes already respected
Write authorization	Both write methods **enforce Edit Mode ownership**: they throw `InvalidOperationException` when there is no session or when rows belong to another employee/user	A reconciler updating other users' rows fails today (**BLOCKER**, `DS-06`)
Settings	`Receiving.BusinessRules.ValidateAllHistoryForLocationReconciliation`, `Receiving.UserPreferences.IgnoredReconciliationLocationsJson`, UI in `Module_Settings.Receiving/Views/View_Settings_Receiving_Reconciliation.xaml`	Settings ownership + migration question (`AR-05`)
UI entry	Mode Selection card → `Enum_ReceivingWorkflowStep.ReconciliationReview` → hosted inside `View_Receiving_Workflow.xaml` (`ReconciliationReviewHost`) as `View_Receiving_Dialog_LocationReconciliationReview`	It is a workflow step, not a real dialog
Interaction model	One-row-at-a-time wizard (Save / Ignore per row), then a summary page	Review.md implies a single "Update Locations" action (`UI-03`, `UI-04`)
SQL files	`Database/InforVisualScripts/Queries/16_GetReceivingLocationEvidence.sql`, `17_GetReceivingLocationTransactionHistory.sql`, `18_GetReceivingLocationTransferMovements.sql`, `18b_GetTransferPairDetails.sql`	`18a_GetTransferCandidatePairs.sql` is referenced by `docs/prompts/reconciliation-location-fix.md` but **does not exist** (`DS-02`)
Precedent	`Module_Reprint` was recently extracted the same way (nav page, `docs/FeatureUpdates/01-FeatureChange-ReprintLabels.md`)	Strongest template for Module_Recon scaffolding
Row schema	`receiving_label_data` has `is_reprint`, `label_number`, `part_skid_sequence`, `part_skid_total`, quality-hold flags, `heat`, `initial_location`. **No print timestamp or print audit column**	"Was a label printed?" cannot be answered today (QoL doc feature A2)
Transfer sign convention	`INVENTORY_TRANS` transfers are stored as **reciprocal positive rows**; lower `TRANSACTION_ID` is source, higher is destination	Answers `Review.md` §9.3 and §9.5 partially

2. Scope And Module Boundary (SC)

SC-01 — What is the module called?

BLOCKER for spec naming.
`Review.md` says `Module_Recon`. Every other module in the repo uses a full word (`Module_Receiving`, `Module_Reporting`, `Module_Reprint`, `Module_Dunnage`, `Module_Volvo`).

- A) `Module_Recon` exactly as written
- B) `Module_Reconciliation`
- C) `Module_Receiving_Reconciliation`

Recommendation: B. Namespace, folder, nav tag, DI extension, and spec title all follow the folder name, so pick once. Abbreviations age badly in a 15-module solution.

SC-02 — Does `GetRecommendedLocationsAsync` move?

`ViewModel_Receiving_LoadEntry` calls `GetRecommendedLocationsAsync` during the normal receiving entry flow (suggesting a location while the operator types). It does not reconcile anything.

- A) Split the interface: recommendation stays in `Module_Receiving`, reconcile methods move
- B) Move the whole interface; `Module_Receiving` takes a dependency on the new module
- C) Duplicate the logic

Recommendation: A. B inverts the dependency direction and creates a Receiving → Recon coupling for a typing assist. C violates the module-coupling constraint against copying logic.

SC-03 — Is `Module_Recon` allowed to reference `Module_Receiving` types?

Reconciliation operates on `Model_ReceivingLoad`, reads `receiving_label_data` / `receiving_history`, and calls `IService_MySQL_Receiving`.

- A) New module references `Module_Receiving` models and services directly
- B) Move `Model_ReceivingLoad` (and its 60+ properties) to `Module_Core`
- C) New module declares its own slim DTO and maps from Receiving models
- D) New module gets its own DAO/contracts and never touches `Module_Receiving`

Recommendation: D for data access, C for models. `Model_ReceivingLoad` is a session/UI model with `ObservableObject`, `JsonIgnore`, and `Visibility` properties; hoisting it into `Module_Core` pollutes the foundational layer. A slim `Model_Recon_*` row type keeps the boundary explicit and makes the new module's tests trivial to author.

SC-04 — Is reconciliation still a Receiving workflow step?

Today it is `Enum_ReceivingWorkflowStep.ReconciliationReview = 12`, hosted inside `View_Receiving_Workflow`.

- A) Remove the enum member and the workflow host entirely
- B) Keep the enum member (value reserved) but never navigate to it
- C) Keep a thin receiving entry point that navigates to the new module

Recommendation: A, plus explicitly deleting the card and command from `ViewModel_Receiving_ModeSelection`. Leaving a dead enum value invites accidental reuse. If any persisted session state stores the enum numerically, confirm before deleting (`LG` risk noted in `TS-06`).

SC-05 — Where does the feature live in navigation?

`MainWindow.xaml` nav tags: `ReceivingWorkflowView`, `DunnageLabelPage`, `VolvoShipmentEntry`, `VisualMacroTester`, `ReportingMainPage`, `ShipRecToolsPage`, `ReprintLabelsPage`,

`ScannerMainPage`, `AppDocumentation`.

- A) New top-level nav item (tag e.g. `ReconPage`), placed next to `ReprintLabelsPage`
- B) Under `ShipRecToolsPage`
- C) Under `ReportingMainPage`

Recommendation: A. It is a distinct operational workflow with its own module, and the `Module_Reprint` extraction set exactly this precedent. Confirm the exact insertion order and label text ("Reconcile Saved Locations" vs "Receiving Reconciliation", see `UI-01`).

SC-06 — Does the old Mode Selection card get removed?

- A) Remove the card, command, and accessibility name
- B) Leave it as a shortcut that opens the new module

Recommendation: A. Two entry points to one workflow doubles the place where permissions, date scope, and telemetry can diverge.

SC-07 — Which artifacts count as "reconcile-specific"?

`Review.md` §2 asks to delete reconcile-specific docs, code, memories, tests, and SQL. The candidate list is in Section 8. Confirm the boundary, especially for query `16` / `17`, which back the *entry-time recommendation* feature, not reconciliation.

SC-08 — Is the new module Receiving-only, or Receiving + Dunnage + Volvo?

`Review.md` only describes Receiving. Dunnage and Volvo have their own label tables and history tables.

- A) Receiving only; leave the door open later
- B) Design the module with pluggable per-source providers now (like `Module_Reprint` did)

Recommendation: A now, B only if a second source is actually requested. `Module_Reprint`'s provider abstraction was justified by three simultaneous sources; one source does not need it yet, but the seam should be named (for example `IModel_ReconSource`) so it is cheap to add.

SC-09 — What is the user-facing feature name?

- A) "Reconcile Saved Locations from InforVisual" (current)
- B) "Receiving Reconciliation"
- C) "Location Reconciliation"

Recommendation: B in navigation, retaining "from Infor Visual" as subtitle text.

3. Scaffolding And Architecture (AR)

AR-01 — DI registration shape

`Infrastructure/DependencyInjection/ModuleServicesExtensions.cs` holds the `AddModuleServices` chain; `Module_Reprint` added a grouped registration extension.

- A) Add `AddReconModule(this IServiceCollection)` in the DI folder and call it from the chain
- B) Inline the registrations into `AddModuleServices`

Recommendation: A. `copilot-instructions.md` forbids registering app services in `App.xaml.cs`, and a grouped extension keeps the new module's wiring in one reviewable place.

AR-02 — Contract and type naming

- A) `IService_Recon_LocationReconciliation`, `Model_Recon_*`, `ViewModel_Recon_*`, `View_Recon_*`

- B) Keep `ReceivingLocationReconciliation`-style names inside the new module
- C) Short names such as `IReconService`

Recommendation: A. It matches the repo's `Module_Prefix_TypeName` convention and keeps grep results unambiguous after the split.

AR-03 — Does the new module get its own DAO?

- A) Yes — new DAO(s) calling dedicated SPs
- B) No — consume the existing `IService_MySQL_Receiving` facade

Recommendation: A for writes and reads, but **only if** `DS-06` (ownership) requires new SPs. If existing SPs are reused as-is, B is acceptable short term but re-introduces cross-module reach-through.

AR-04 — Settings ownership and migration

Two settings drive today's behavior. Both live under `Receiving.*` keys and are rendered by `Module_Settings.Receiving`.

- A) New `Recon.*` keys with a one-time copy from the `Receiving.*` keys
- B) Keep the `Receiving.*` keys and render them in the new module's settings page
- C) Keep the keys and keep the settings page in `Module_Settings.Receiving`

Recommendation: A. Duplicated keys with no migration silently reset every user's ignored locations, which changes reconciliation results on day one.

AR-05 — Where does the settings UI live?

- A) `Module_Settings.Recon` (new settings module)
- B) `Module_Settings.Receiving` (existing)
- C) Inline settings panel inside the new working module

Recommendation: B for this refactor. A follows the one-settings-module-per-module pattern but adds a second project and a settings-manifest change; that is a second refactor, not part of a feature extraction.

AR-06 — Header title and back navigation

`MainWindow` header resolves titles through `IViewModel_HeaderTitleProvider` with fallback titles, and back navigation runs through `IService_HeaderBackNavigation`.

- A) New module viewmodels implement `IViewModel_HeaderTitleProvider` and use the shared back service
- B) Rely on fallback titles

Recommendation: A, and register the new module in the accent-brush theming (`Module_Core/Themes/ModuleAccentBrushes.xaml`). Question: should the new module get a distinct accent color, or inherit the Receiving accent? **Recommendation:** distinct accent, matching how `Module_Reprint` reads as its own area.

AR-07 — Do we add a schema migration?

Depends on `DS-07` (run audit) and the QoL document. If no new columns/tables are approved, this refactor is migration-free.

Recommendation: keep the refactor migration-free; put audit tables behind a separate, explicitly approved change.

AR-08 — Long-running execution model

`includeAllHistory` with `StartDate = 2000-01-01` can scan a very large history.

- A) Keep the current single `await` with an indeterminate progress banner
- B) Chunked/background run with progress reporting and cancellation
- C) Hard row cap with a "narrow your date range" message

Recommendation: B + C. A user-selected date range plus an "All history" preset without a cap is a performance incident waiting to happen.

AR-09 — Where do the Visual SQL files live after the move?

- A) Leave in `Database/InforVisualScripts/Queries/` with existing numbers
- B) Move to `Database/InforVisualScripts/Queries/Recon/`
- C) Renumber

Recommendation: A + B (move into a `Recon/` subfolder without renumbering). Renumbering breaks every cross-reference in docs and prompts for no functional gain.

AR-10 — Logging and run identity

- A) Existing `IService_LoggingUtility` messages only
- B) Add a correlation/run id to every log line for one reconciliation run

Recommendation: B. Multi-row, multi-group runs are otherwise impossible to reconstruct from logs.

4. UI And UX (UI)

UI-01 — Launcher surface

Recommendation: one top-level nav page (see `SC-05`). No card inside Receiving.

UI-02 — Date range control

Today: a single boolean. `Review.md` requires a user-supplied range.

- Which control: two `CalendarDatePicker`s, a `DateRangePicker`-style pair, or preset buttons?
- Which presets: Today, Yesterday, Last 7 days, Last 30 days, This month, Custom, All history?
- What is the default on open?
- Is the selected range persisted per user, or reset each visit?
- Maximum allowed span (guard rail for `AR-08`)?

Recommendation: quick presets + custom pair; default = Today; **not** persisted (matching `Module_Reprint`'s "date range is not persisted and defaults to Today"); max span 31 days unless "All history" is explicitly chosen.

UI-03 — Preview vs apply

`Review.md` step 4 says the user clicks "Update Locations". Today the flow is preview → per-row Save/Ignore → summary.

- A) Keep preview → review → apply (safe, current)
- B) Single "Update Locations" bulk action with a confirmation dialog
- C) Preview → grid with checkboxes → "Apply Selected"

Recommendation: C. It satisfies `Review.md`'s single-button intent while keeping a visible per-row decision, and it matches the `Module_Reprint` grid-with-checkboxes pattern reviewers already know.

UI-04 — Review screen shape

- A) One-at-a-time wizard (current `ViewModel_Receiving_LocationReconciliationReview`)
- B) Grid with row selection and a detail pane

Recommendation: B. At more than ~10 rows the wizard becomes a click marathon, and the current model already carries everything a grid needs (`ExistingLocation`, `ProposedLocation`, `MovedByUserId`, `MovedAt`, `Resolution`, `Details`).

UI-05 — Which columns?

Candidate set from `Model_ReceivingLocationReconciliationItem` + `Model_ReceivingLoad`: part, description, PO, line, load/skid number, quantity + UOM, current location, proposed location, Visual user, transfer time, transfer count, source locations, resolution, reason.

- Which are visible by default?
- Is a column chooser required (as in `Module_Reprint`)?
- Should `Details` be a tooltip, a detail pane, or a wrapping column?

Recommendation: default = part, description, PO, line, qty, UOM, current, proposed, resolution; detail pane for reasons and evidence; column chooser only if requested.

UI-06 — Filtering and sorting

- Filter by resolution (Updated / Ambiguous / NotFound / Skipped / IgnoredBySettings)?
- Filter by part, PO, user, source (Current Labels vs History)?
- Default sort?
- Group rows by part + PO?

Recommendation: filter by resolution and part/PO text search; group by part + received date (mirrors the algorithm's group key, which makes "why are these 5 rows tied together" obvious).

UI-07 — How are exceptions surfaced?

- A) Inline in the same grid with a red row background
- B) Separate "Needs Attention" tab
- C) Summary counts only, with export

Recommendation: B. Unresolved rows are the actionable output of the module; burying them in a mixed grid is why the current summary gets closed without reading it.

UI-08 — Progress, cancellation, and re-run

- Is there a visible row/group counter during a run?
- Is cancel required?
- Can a run be re-run without closing the page?

Recommendation: counter + cancel, in-place re-run. Reconciliation is naturally re-runnable because `Unchanged` is a first-class outcome.

UI-09 — Empty and failure states

Define text for: no rows in range, all rows unchanged, all rows skipped, Visual unreachable, MySQL unreachable, range too wide. Should an empty result be a page-level state or a dialog?

Recommendation: page-level empty state with a "widen the date range" action; Visual failure as an inline retry banner rather than a modal that drops the user back to Mode Selection (the current behavior).

UI-10 — Summary content

Must the summary show: scanned count, updated count, unchanged count, unresolved count, skipped count, error count, run duration, date range used, and the Visual user attribution?

Recommendation: yes to all, plus an export action (see QoL `E4`) and a copyable run id (`AR-10`).

UI-11 — Manual override

Should the user be able to type a location for an `Ambiguous` row and force-save it?

- A) No — only transfer-evidenced updates are writable
- B) Yes, as a separate explicitly-labelled manual action

Recommendation: B, but only as a separate action with a distinct resolution value (for example `ManuallySet`) so audit and reporting can tell evidenced updates from typed ones. Do not let it reuse the `Updated` path.

UI-12 — Accessibility and keyboard

WinUI rules in this repo require `x:Bind`, `AutomationProperties.Name` on interactive controls, and keyboard access.

- Are keyboard shortcuts required (next/previous row, apply, re-run)?
- Does the grid need a screen-reader summary count?

Recommendation: yes to both; `Module_Reprint` and `View_Receiving_ModeSelection` already model the pattern.

UI-13 — Where does the "reprint" concern appear?

If QoL features around label print integrity are adopted, do they render inside this module, in `Module_Reprint`, or in `Reporting`?

Recommendation: detection and disposition in Module_Recon; the actual reprint action deep-links to `Module_Reprint` rather than duplicating its queue logic.

5. Reconciliation Logic (LG)

LG-01 — What is the grouping key? **BLOCKER**

Current: `(PartID, ReceivedDate.Date)` — PO is used only as a softer key inside `SelectRelevantMovements`.
`Review.md`: match on `PO + Part Number`, with total quantity per combination.

- A) `(PartID, ReceivedDate)` — current behavior
- B) `(PO, PartID)` — Review.md literal reading
- C) `(PO, PartID, ReceivedDate)`
- D) `(PO, PartID, POLine, ReceivedDate)`

Why it matters: the group key decides which rows compete for the same transfer quantity. A part received twice in one day from two POs is one bucket under A and two under B/C. Under C, a part received on two days from one PO is two groups, which is what the ±2 day window and the receipt-date-scoped strict check already assume.

Recommendation: C. It preserves the date scoping the algorithm depends on and adds PO isolation that `Review.md` explicitly asks for. If C is chosen, confirm that two receipts of the same PO+part on the *same* day should still share one allocation group (they do today).

LG-02 — Is there a "receiver record" to query? **BLOCKER**

`Review.md` §5.1 says to retrieve the receiver record/transaction for the PO and part number. No `RECEIVER` / `RECEIVER_LINE` query exists in the repo. Today the "receiver" is the MTM saved row itself (`weight_quantity` , `initial_location` , `received_date`).

- A) MTM rows are the receiver proxy; no Visual receiver query is added
- B) Add a Visual `RECEIVER` / `RECEIVER_LINE` query and reconcile against true ERP receipts
- C) Both: MTM rows drive allocation, Visual receiver quantity is used as a cross-check

Recommendation: A for the extraction; C as a follow-up in the QoL document. Adding ERP receipt queries is a new data-source project, not a module move, and `Review.md` §7.9 forbids unrelated cleanup inside this refactor.

LG-03 — Is a current-inventory comparison required?

`Review.md` §5.2 requires comparing Receiving App locations with the material's *current* Visual locations and skipping when they already match. The preview path does not do this today; it goes straight to transfer evidence.

- A) No — transfer evidence is authoritative (current)
- B) Yes — add a first-pass current-inventory check (`CR_PART_LOCATION` / evidence query `16`) that short-circuits to `Unchanged`
- C) Yes, but only as a reportable warning

Recommendation: B only if a real "already correct" false-positive problem exists. B adds a Visual round-trip per part and introduces a second definition of "correct location"; A is cheaper and already validated by tests. If B is chosen, define whether a mismatch between *current inventory* and *transfer destination* blocks the update.

LG-04 — Which quantity gate is authoritative? **BLOCKER**

Two different gates exist:

1. Current: for each destination bucket, `sum(allocated row quantities) == transfer qty to that bucket in the receipt-date window` .
2. `Review.md` : for the receiver, `sum(relevant transaction qty) == received qty` , then update every matching line to the used location.

- A) Keep gate 1 only (bucket-level)
- B) Gate 2 only (receiver-level)
- C) Both: receiver-level total must match first, then bucket-level allocation must also match
- D) Gate 1, plus a receiver-level pre-check that is reportable but not blocking

Why it matters: gate 2 cannot express a split receipt (16,400 to one location, 8,840 to another) without inventing an allocation rule anyway. Gate 1 already produces the deterministic split that `Review.md` 's example needs.

Recommendation: C. Keep bucket-level allocation as the update authority and add the receiver-level total as a pre-gate so a receipt that cannot possibly reconcile is rejected before allocation runs. Note that under C, `Review.md` 's example (25,240 received, 16,400 + 8,840 transferred) reconciles, while a receipt with un-transferred remainder becomes `Ambiguous` — confirm that is desired.

LG-05 — Which quantity field represents a row?

`receiving_label_data` has both `quantity` (int, rounded) and `weight_quantity` (decimal 18,2). Current allocation uses `WeightQuantity` .

- A) `WeightQuantity` (current)
- B) `Quantity`
- C) `WeightQuantity` , falling back to `Quantity` when null

Recommendation: C. It matches what the labels show and removes the null-row edge case.

LG-06 — Unit of measure handling

Rows in EA and rows in LB can coexist for different parts; are there parts where both appear?

- Should reconciliation refuse to allocate rows whose UOM differs from the transfer evidence?
- Should a mixed-UOM group be blocked outright?

Recommendation: block allocation when the group's UOM set has more than one value, and mark those rows `Ambiguous` with an explicit reason. Silent cross-UOM arithmetic is the worst possible outcome for a weight-based receiving app.

LG-07 — Exact equality or tolerance?

`Review.md` §9.4 asks. Current code uses exact `decimal` equality.

- A) Exact equality
- B) Absolute tolerance (for example ±0.01) for weight rounding
- C) Percentage tolerance

Recommendation: B with a named constant and a settings key, defaulting to exact. Weights are entered by humans and stored at two decimals; a 0.01 epsilon prevents spurious `Ambiguous` rows without weakening the gate meaningfully.

LG-08 — Transfer pairing assumption

Current: reciprocal positive rows; lower `TRANSACTION_ID` = source, higher = destination.

- Keep as-is?
- Is it worth validating in the query (for example, reject pairs where both rows are inbound to the same location)?

Recommendation: keep the assumption (documented as an invariant in `docs/prompts/reconciliation-location-fix.md` Fix 5), but add a guard that drops same-location pairs (already partially present) and log the count of dropped pairs.

LG-09 — Chained move rule

Current: net destination after chained moves; staging retains quantity if not fully forwarded.

- Keep?
- Does partial retention in staging make staging an eligible destination bucket (current: yes)?

Recommendation: keep exactly as documented in `docs/features/receiving/location-reconciliation-process.md`. If behavior changes, that document must change in the same commit.

LG-10 — Date windows

Three different windows exist today:

- Discovery: receipt date ±2 days
- Strict validation: receipt date and receipt date + 1
- Row selection: `includeAllHistory ? 2000-01-01 : today` → today + 1
- Should discovery stay ±2 days?
- Should the strict window stay at +1 day?
- For a user-selected range, is the range applied to **row selection only**, or also to the discovery window around each row's receipt date?

Recommendation: range applies to row selection; discovery and strict windows stay anchored to each row's receipt date. Both windows become settings keys so they can be tuned without a code change (confirm this is acceptable given `AR-04`).

LG-11 — Reprint rows (is_reprint = 1)

A reprint re-queues a history row into `receiving_label_data`. That row carries the same part, PO, quantity, and location as a row that may still exist in history. Including it risks double-counting the same physical skid against one transfer quantity.

- A) Include reprint rows like any other row
- B) Exclude reprint rows from reconciliation, but list them as informational
- C) Include, but reconcile in a separate group so they cannot consume the original row's transfer quantity

Recommendation: B. A reprint is a label document, not a new physical receipt. This is a **BLOCKER** for correctness if reprints are common in production.

LG-12 — Idempotency and re-runs

Running twice over an overlapping range must not produce a second update.

- A) Already-current rows resolve to `Unchanged` (expected today)
- B) Also detect rows updated by a previous reconciliation run and skip them with a distinct resolution

Recommendation: A plus B (`AlreadyReconciled`), which requires an audit trail (`DS-07`).

LG-13 — Non-PO rows

Current: `Skipped` with a reason; hidden from the attention list.

- Keep skipping?
- Should they appear in the exported output as skipped, even though not actionable?

Recommendation: keep skipping, still exportable.

LG-14 — Ignored locations

Current: destination buckets in the ignored set make rows `IgnoredBySettings`.

- Should ignore apply to destination only, source only, or both?
- Should the ignore list be per-user or per-workstation?

Recommendation: destination only (matches the current reason text), per-user (it is stored under user preferences today).

LG-15 — `NotFound` semantics BLOCKER-ish

Current: no qualifying transfers → row is treated as "saved location is probably still correct" and is **not** surfaced in the attention list. `Review.md` §9.9 asks what should happen.

- A) Silent (current)
- B) Reported but not actionable
- C) Flagged as needs-review

Recommendation: B. Silently hiding the single most common outcome ("we found no transfers") makes the module look like it did nothing when it likely means the material never moved — the user should still see the count.

LG-16 — Over-transfer and under-transfer

Define the outcome when transfer total is greater than, or less than, the received quantity.

- Under: current → `Ambiguous`
- Over: current → allocation succeeds if buckets cover rows; the surplus is unallocated

Recommendation: under stays `Ambiguous` ; over produces `Ambiguous` with an explicit "surplus quantity" reason, because a surplus usually means the movement search picked up a neighboring receipt (`docs/prompts/reconciliation-location-fix.md` Fix 2 exists precisely because of that).

LG-17 — Rounding and decimal arithmetic

Sources are `decimal(18,2)` ; sums are exact. Confirm no intermediate `double` / `float` conversions and that display formatting never feeds back into comparisons.

Recommendation: compare in `decimal` only; format with `CultureInfo.InvariantCulture` in messages (already the pattern).

LG-18 — What "assy" means

`Review.md` §9.5 asks about the handwritten phrase "total Qty of assy". Current analog: the sum of transfer quantities for the group.

Recommendation: confirm the intended meaning is "total quantity transferred/consumed". If it means "assembly quantity" from a work-order context, that is a different Visual table (`WIP_ISSUE_DETAIL`) and a materially larger feature. **This answer changes the data source.**

6. Data Sources And Schema (DS)

DS-01 — MySQL read endpoints

Confirm the read surface for the new module:

- Current rows: `IService_MySQL_Receiving.GetCurrentLabelDataAsync()` → `sp_Receiving_LabelData_GetAll`
- History rows: `GetAllReceivingLoadsAsync(start, end)` → `Dao_ReceivingLoad.GetAllAsync`

Questions:

- Are range bounds inclusive of `start` and exclusive of `end` ? (Today: `Today` → `Today + 1` .)
- Does the history query filter on `received_date` , `transaction_date` , or `created_at` ? `Review.md` §9.8 asks which date controls inclusion.
- Should the new module call these existing methods or get new SPs with explicit date parameters?

Recommendation: new SPs with an explicit `(StartDate, EndDate, IncludeReprints)` shape, so the date column is documented in the SP contract instead of implied by the caller. Confirm the date column (`received_date` recommended).

DS-02 — Visual query inventory

Current files: `16` (evidence), `17` (transaction history), `18` (transfer movements), `18b` (pair details). `18a_GetTransferCandidatePairs.sql` is referenced by `docs/prompts/reconciliation-location-fix.md` but is absent — the candidate-pair CTE lives inside query `18` .

- Should the 18a/18b split be completed as part of this refactor?
- Are queries `17` and `16` still used by anything after the split? (`16` backs `GetRecommendedLocationsAsync` ; `17` appears unused by the current service — verify.)

Recommendation: complete the 18a/18b split (it is documented as intended and makes the candidate step testable in SSMS), and delete query `17` only after confirming zero callers.

DS-03 — Are `RECEIVER` / `RECEIVER_LINE` needed?

Tied to `LG-02`. If yes, the query must be authored per `.github/instructions/database/infor-visual-query-authoring.instructions.md` (research-first against the CSV reference files) and reviewed as a separate data-source change.

DS-04 — Performance guardrails on Visual

`INVENTORY_TRANS` is roughly 3.5M rows. Current query `18` filters by `PART_ID` and a date window.

- Is a per-part loop acceptable, or is a set-based multi-part query required for a month-long range?
- What is the maximum acceptable query time before the module degrades?

Recommendation: keep per-part set-based queries (one part per call, already the shape) and cap the date range (`UI-02`). Add `TOP` protection on the candidate-pair step.

DS-05 — Read-only enforcement

`ApplicationIntent=ReadOnly` is already required in the Visual connection string. Confirm nothing in the new module writes to Visual, including no "mark as audited" flag in ERP.

Recommendation: keep read-only; audit lives in MySQL only.

DS-06 — Write authorization model **BLOCKER**

`UpdateReceivingLoadsAsync` and `UpdateCurrentLabelDataAsync` require an active session **and** reject rows owned by another employee or Windows user (`GetUnauthorizedLoads` → `InvalidOperationException`). A reconciliation tool that fixes history rows created by other operators will throw on the first foreign row.

- A) Reconciliation only updates rows owned by the current user
- B) Reconciliation uses a privileged path that bypasses ownership (supervisor/admin only)
- C) Reconciliation proposes changes but writes nothing; an owning user applies them later

Why it matters: A silently hides most of the problem the module exists to solve; C turns the module into a report; B requires new SPs, a role check, and an audit trail. This decision shapes the service contract, the SPs, and the tests.

Recommendation: B with an explicit role gate and a mandatory audit row per change. If role infrastructure is not ready, ship C first — never ship A without telling the user why rows were skipped.

DS-07 — Do we persist reconciliation runs?

- A) No persistence; logs only
- B) New `recon_run` table (user, start/end, range, counts, duration)
- C) B plus `recon_change` per-row audit (old location, new location, resolution at write time, evidence ids, Visual user, timestamp)

Recommendation: C. Without C, "who changed this location and why" cannot be answered, `LG-12` is impossible, and the QoL disposition features are blocked. This needs explicit schema approval.

DS-08 — Ignored-location storage

Stored as a JSON array string under a settings key.

- Keep JSON, or normalize into a table?

Recommendation: keep JSON for this refactor; normalize only if the QoL features need per-location metadata (who ignored it, when, why).

7. Testing (TS)

TS-01 — Test placement and naming

Existing: `MTM_Receiving_Application.Tests/Unit/Module_Receiving/{Services,ViewModels}` .
Questions: `Unit/Module_Recon/...` ? Naming `MethodName_ShouldResult_WhenCondition` (repo rule), FluentAssertions, `TEST-` data prefix for integration.

Recommendation: mirror `Unit/Module_Reprint/...` .

TS-02 — Which existing tests move?

- `Service_ReceivingLocationReconciliationTests.cs` — the `GetRecommendedLocationsAsync` test stays in Receiving; all allocation tests move.
- `ViewModel_Receiving_LocationReconciliationReviewTests.cs` — moves.
- `ViewModel_Receiving_LoadEntryTests.cs` — stays, but its mock target changes to the split interface.

Recommendation: as above; split the service test file rather than copying it.

TS-03 — Is there a Visual database test harness?

Current integration tests target MySQL stored procedures. The Visual SQL is validated manually.

- A) Visual queries stay manually validated (SSMS) and are covered by unit tests over fakes
- B) Create a read-only Visual integration harness

Recommendation: A. Do not add a live ERP dependency to the test suite.

TS-04 — Which fixtures are golden cases?

Keep `MMC0000650` (five skids, 12000/5000/6350 split) and `docs/features/receiving/location-reconciliation-process.md` 's acceptance example as named regression fixtures.

Recommendation: yes, name them (`GoldenCase_FiveSkidSplit`) so a future reader can trace the numbers back to the business example.

TS-05 — Required coverage from `Review.md` §7.10

Map each to an explicit test:

Review.md requirement	Test
Matching locations	`..._ShouldReturnUnchanged_WhenProposedEqualsSaved`
Mismatched locations	`..._ShouldProposeDestination_WhenSavedDiffersFromTransfer`
Quantity equality	`..._ShouldUpdate_WhenBucketTotalEqualsAllocation`
Quantity mismatch	`..._ShouldReturnAmbiguous_WhenBucketTotalDiffers`
Multiple destinations	`..._ShouldAllocateAcrossBuckets_WhenReceiptSplits`
Date-range boundaries	`..._ShouldExcludeRow_WhenReceiptOutsideRange`
Multiple receipts same part	`..._ShouldIsolateGroups_WhenSamePartHasTwoReceiptDates`

Recommendation: add: reprint exclusion (`LG-11`), ignored destination (`LG-14`), cross-pass merge (`LG-12`), UOM conflict (`LG-06`), ownership rejection (`DS-06`), cancellation (`AR-08`).

TS-06 — Enum and navigation regression

If `Enum_ReceivingWorkflowStep.ReconciliationReview` is deleted, confirm nothing persists the numeric value (session restore, saved workflow state).

Recommendation: grep for numeric persistence before deleting; if found, keep the member reserved with a comment instead.

TS-07 — ViewModel tests

Is the new module's viewmodel expected to have unit tests (date range validation, filtering, selection, apply command guard)?

Recommendation: yes — model state, not XAML.

TS-08 — Validation commands

`dotnet build MTM_Receiving_Application.slnx` and `dotnet test MTM_Receiving_Application.Tests/MTM_Receiving_Application.Tests.csproj`. Confirm the spec should require a focused filter run (for example `--filter FullyQualifiedName~Module_Recon`) plus a full-suite no-new-failures check, as `Module_Reprint` did.

8. Migration And Cleanup (MG)

MG-01 — Artifact disposition table

Confirm each row's disposition (`MOVE` / `DELETE` / `KEEP SHARED`).

Artifact	Proposed	Notes
`Module_Receiving/Services/Service_ReceivingLocationReconciliation.cs`	MOVE	Split: recommendation met stays
`Module_Receiving/Contracts/IService_ReceivingLocationReconciliation.cs`	SPLIT	Keep recommendation con in Receiving
`Module_Receiving/Models/Model_ReceivingLocationReconciliationItem.cs`	MOVE	Rename per `AR-02`
`Module_Receiving/Models/Model_ReceivingLocationReconciliationSummary.cs`	MOVE	Rename per `AR-02`
`Module_Receiving/ViewModels/ViewModel_Receiving_LocationReconciliationReview.cs`	MOVE	Replaced by grid viewmod (`UI-04`)
`Module_Receiving/Views/View_Receiving_Dialog_LocationReconciliationReview.xaml(.cs)`	MOVE	Not a real dialog; drop the `Dialog_` token
`Module_Receiving/Views/View_Receiving_Workflow.xaml(.cs)`	EDIT	Remove `ReconciliationReviewHost`
`Module_Receiving/Views/View_Receiving_ModeSelection.xaml`	EDIT	Remove the reconcile card
`Module_Receiving/ViewModels/ViewModel_Receiving_ModeSelection.cs`	EDIT	Remove `ReconcileSavedLocationsCo`
`Module_Core/Models/Enums/Enum_ReceivingWorkflowStep.cs`	EDIT	See `SC-04` / `TS-06`
`Module_Receiving/Settings/ReceivingSettingsKeys.cs` (2 keys)	MOVE + migrate	See `AR-04`
`Module_Settings.Receiving/Views/View_Settings_Receiving_Reconciliation.xaml(.cs)`	MOVE	See `AR-05`
`Module_Settings.Receiving/ViewModels/ViewModel_Settings_Receiving_*` (recon parts)	MOVE	Verify which VM hosts it
`Database/InforVisualScripts/Queries/18*.sql`	MOVE	See `AR-09`
`Database/InforVisualScripts/Queries/16*.sql`	KEEP SHARED	Backs entry-time recommendation

Artifact	Proposed	Notes
`Database/InforVisualScripts/Queries/17*.sql`	DELETE if unused	Verify zero callers first
`MTM_...Tests/Unit/Module_Receiving/Services/Service_ReceivingLocationReconciliationTests.cs`	SPLIT	See `TS-02`
`MTM_...Tests/Unit/Module_Receiving/ViewModels/ViewModel_Receiving_LocationReconciliationReviewTests.cs`	MOVE	
`docs/features/receiving/location-reconciliation-process.md`	MOVE	Rewrite for the new module
`docs/prompts/reconciliation-location-fix.md`	KEEP as history	Or move under the new module's docs
`Module_Receiving/Proposed_UI_Changes.xaml.md` / `.html`	VERIFY	Determine whether reconci specific
`Infrastructure/DependencyInjection/ModuleServicesExtensions.cs`	EDIT	See `AR-01`
`MainWindow.xaml` / `MainWindow.xaml.cs`	EDIT	Nav item, route, search ter

MG-02 — Do docs move or get rewritten?

The existing process doc describes the *implemented* algorithm, which is richer than `Review.md`. A naive rewrite loses the chained-move and partial-move rules.

Recommendation: move the file, keep every behavioral rule, and add a new "Selected date range" section. Never let the spec restate the algorithm from `Review.md` alone.

MG-03 — What happens to `.serena/memories`?

`Review.md` says remove reconcile-related memories. Which memory files qualify, and should they be rewritten instead of deleted?

Recommendation: rewrite in place (`infor-visual-inventory-discrepancy-playbook` is broader than reconciliation); delete only memories that name the Receiving path explicitly.

MG-04 — Removal ordering

Should the new module land before the old UI is removed (as `Module_Reprint` did) to avoid a capability gap in a shipped app?

Recommendation: yes — land the module first, remove the old surface in the same PR but as the last step, and verify the app still builds between steps.

9. Non-Functional (NF)

NF-01 — Performance targets

Rows scanned per run, maximum runtime for a 31-day range, acceptable Visual query latency, and the maximum row count the grid must stay responsive with.

Recommendation: state explicit numbers in the spec (for example ≤5,000 rows scanned, ≤60 s for 31 days, grid virtualized above 500 rows).

NF-02 — Error handling and retry

- Retry policy for Visual timeouts?
- Partial-failure behavior when one part's query fails: fail the run, or mark that group `Error` and continue?
- Current behavior: per-group `Error` items, run continues.

Recommendation: keep per-group isolation; add a single bounded retry per Visual call and a run-level `Error` summary.

NF-03 — Error escalation boundary

MVVM rules require viewmodels to surface errors through `IService_ErrorHandler` and keep business logic in services. Confirm the new module follows the same split and does not add its own dialogs.

NF-04 — Settings caching

`IgnoredReconciliationLocationsJson` is read once per run today. If it becomes editable inside the module (`UI-11`), confirm invalidation behavior.

NF-05 — Privacy

Row data includes employee numbers and Windows usernames. Confirm:

- which of those may be logged,
- whether the audit trail (`DS-07`) may store them,
- whether exports (`QoL E4`) are restricted.

Recommendation: store employee number in audit, never log Windows usernames at Information level.

NF-06 — Localization and text

Are all new UI strings added to the settings text store (the pattern used by `ReceivingSettingsKeys.UiText`) or hard-coded?

Recommendation: follow the existing `UiText` key pattern for anything user-visible.

10. Recommended Defaults

If every recommendation above is accepted, the spec resolves as follows.

1. Module name `Module_Reconciliation` (`SC-01`).
2. Recommendation method stays in `Module_Receiving` ; reconcile methods move (`SC-02`).
3. New module owns its data access and uses slim `Model_Recon_*` row types (`SC-03` , `AR-03`).
4. Reconciliation leaves the Receiving workflow and the Mode Selection card is deleted (`SC-04` , `SC-06`).
5. New top-level nav page with its own accent and header provider (`SC-05` , `AR-06`).
6. Grouping key becomes `(PO, PartID, ReceivedDate)` (`LG-01`).
7. Bucket-level allocation stays authoritative; receiver-level total becomes a pre-gate (`LG-04`).
8. Reprint rows are excluded and reported (`LG-11`).
9. `NotFound` is counted and reportable but not actionable (`LG-15`).
10. Quantity uses `WeightQuantity` with `Quantity` fallback; exact decimal compare with a 0.01 epsilon setting (`LG-05` , `LG-07`).
11. Mixed-UOM groups are blocked with a reason (`LG-06`).
12. Date range = presets + custom, default Today, not persisted, 31-day cap (`UI-02` , `AR-08`).
13. Preview → grid with checkboxes → apply selected, plus a separate Needs Attention tab (`UI-03` , `UI-04` , `UI-07`).
14. Writes use a privileged, role-gated path with per-row audit; schema change explicitly approved (`DS-06` , `DS-07`).
15. Settings migrate to `Recon.*` keys with a one-time copy; settings UI stays in `Module_Settings.Receiving` for this refactor (`AR-04` , `AR-05`).
16. Visual SQL moves to a `Queries/Recon/` folder without renumbering; the 18a/18b split is completed; query `17` is deleted if unused (`DS-02` , `AR-09`).

17. Tests mirror `Module_Reprint`, golden fixtures named, plus reprint/UOM/ownership/cancellation cases (`TS-*`).

11. Decision Record

Fill this in, or reply "accept all recommendations" to adopt Section 10.

ID	Question	Decision	Decided by	Date
SC-01	Module name			
SC-02	Recommendation method location			
SC-03	Cross-module type strategy			
SC-04	Workflow enum removal			
SC-05	Navigation placement			
SC-06	Mode Selection card removal			
SC-07	Cleanup boundary			
SC-08	Multi-source scope			
SC-09	Feature name			
AR-01	DI registration shape			
AR-02	Type naming			
AR-03	New DAO vs shared facade			
AR-04	Settings key migration			
AR-05	Settings UI location			
AR-06	Header/accent wiring			
AR-07	Migration required			
AR-08	Execution model			
AR-09	SQL file location			
AR-10	Run identity logging			
UI-01	Launcher surface			
UI-02	Date range control			
UI-03	Preview vs apply			
UI-04	Review screen shape			
UI-05	Columns			
UI-06	Filter/group			
UI-07	Exception surfacing			
UI-08	Progress/cancel			
UI-09	Empty/failure states			
UI-10	Summary content			
UI-11	Manual override			
UI-12	Accessibility			

ID	Question	Decision	Decided by	Date
UI-13	Reprint concern placement			
LG-01	Grouping key			
LG-02	Receiver data source			
LG-03	Current-inventory compare			
LG-04	Quantity gate			
LG-05	Row quantity field			
LG-06	UOM handling			
LG-07	Tolerance			
LG-08	Transfer pairing			
LG-09	Chained moves			
LG-10	Date windows			
LG-11	Reprint rows			
LG-12	Idempotency			
LG-13	Non-PO rows			
LG-14	Ignored locations			
LG-15	NotFound semantics			
LG-16	Over/under transfer			
LG-17	Decimal arithmetic			
LG-18	"assy" meaning			
DS-01	MySQL read endpoints			
DS-02	Visual query inventory			
DS-03	RECEIVER usage			
DS-04	Visual performance guardrails			
DS-05	Read-only guarantee			
DS-06	Write authorization			
DS-07	Run/audit persistence			
DS-08	Ignored-location storage			
TS-01..TS-08	Test strategy			
MG-01..MG-04	Cleanup disposition			
NF-01..NF-06	Non-functional targets			